

Asymptotic analysis

CS3230 · Lecture 1

Demo course material, written for OpenPapr

Why count steps?

- Wall-clock time depends on the machine
- Count basic operations instead
- Focus on growth as n becomes large

Input size

- n = number of items, bits, or vertices
- Worst case, best case, average case
- We usually analyse the worst case

Growth rates 1

- $\text{Constant} < \log n < n < n \log n < n^2 < 2^n$
- Constants are dropped
- Lower-order terms are dropped

Growth rates 2

- $\text{Constant} < \log n < n < n \log n < n^2 < 2^n$
- Constants are dropped
- Lower-order terms are dropped

Growth rates 3

- $\text{Constant} < \log n < n < n \log n < n^2 < 2^n$
- Constants are dropped
- Lower-order terms are dropped

Big-O

- $f(n) = O(g(n))$ if $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- An upper bound on growth
- $3n^2 + n$ is $O(n^2)$

Big-Omega

- $f(n) = \Omega(g(n))$ if $f(n) \geq c \cdot g(n)$ for $n \geq n_0$
- A lower bound on growth
- $n \log n$ is $\Omega(n)$

Big-Theta

- Both O and Omega at once
- $5n + 2$ is $\Theta(n)$

Theta is a claim about BOTH bounds

Proving a bound

- Pick constants c and n_0
- Show the inequality for all $n \geq n_0$
- Common mistake: picking c after n_0 depends on it

Little-o and little-omega

- Strict versions of O and Ω
- $f = o(g)$ means f grows strictly slower
- $n = o(n^2)$ but n is not $o(n)$

The three bounds

- $O(g)$: grows no faster than g
- $\Omega(g)$: grows at least as fast as g
- $\Theta(g)$: both

Logarithms

- $\log_a n = \Theta(\log_b n)$ for constants a, b
- So the base does not matter inside O
- $n^{0.001}$ eventually beats $\log n$

Polynomials vs exponentials

- $n^k = o(2^n)$ for any constant k
- $2^n = o(3^n)$
- $n!$ grows faster than 2^n

Sums

- $1 + 2 + \dots + n = \Theta(n^2)$
- $1 + 1/2 + \dots + 1/n = \Theta(\log n)$
- Geometric sums are dominated by the largest term

Practice

- Rank: $n \log n$, $n^{1.5}$, $2^{\log n}$, $\log n!$
- Hint: $\log n! = \Theta(n \log n)$
- Answers in Tutorial 1

Comparing growth rates

- Take the limit of $f(n)/g(n)$ as n grows
- 0 means $f = o(g)$; a constant means Θ
- Infinity means $f = \omega(g)$

Comparing growth rates

- Take the limit of $f(n)/g(n)$ as n grows
- 0 means $f = o(g)$; a constant means Θ
- Infinity means $f = \omega(g)$

Comparing growth rates

- Take the limit of $f(n)/g(n)$ as n grows
- 0 means $f = o(g)$; a constant means Θ
- Infinity means $f = \omega(g)$

Summary

- Count operations, not seconds
- Use O , Ω , Θ precisely
- Next: recurrences